UNIVERSIDADE DO MINHO

Computer Systems Security (SSC) — MCS, 2026–27

Lab report 1: Respostas
Labs 1 + 2

Lab 1 (First Contact) + Lab 2 (Threat Modelling) — um único PDF de respostas

Disciplina	Computer Systems Security (SSC)
Mestrado	MCS, 2026–27
Instituição	Universidade do Minho
Entrega	Lab report 1 (única entrega do bloco: cobre Labs 1 e 2)
Aluno	Hugo Soares a107293
Data	28 de outubro de 2026

PDF de respostas: P1/P2 e Exercícios dos Labs 1 e 2. O Lab 1 não tem relatório próprio; as suas respostas vão na Secção Lab 1.

Nenhuma exploração foi realizada.

Conteúdo

1 Lab 1 — Respostas **3**

1.1 Ex. 7 — Inventário da superfície de ataque 3

1.2 Preparação do Lab 1 (P1, P2) 3

2 Lab 2 — Respostas **4**

2.1 Perguntas de preparação (P1, P2) 4

2.2 P1 — Quais são os quatro elementos de um diagrama de fluxo de dados? 4

2.3 P2 — O que é que STRIDE dá que uma lista ad-hoc de "problemas" não dá? 4

3 Parte A — O atacante e os ativos **4**

4 Parte B — Diagrama de fluxo de dados da máquina **6**

5 Parte C — STRIDE, elemento a elemento **7**

6 Parte D — Uma árvore de ataque **10**

7 Parte E — Priorizar e medir **11**

1 Lab 1 — Respostas

1.1 Ex. 7 — Inventário da superfície de ataque

Tabela 1 Inventário da superfície de ataque — Lab 1, Exercício 7.

Item de superfície	Dimensão	Corre / é detido por	Quem alcança ou modifica	Pergunta em aberto
Serviço na porta 8081	Serviço em escuta	Conta de serviço não privilegiada (`svc-report`)	Alcançável em todas as interfaces; a conta de serviço é a detentora	A lógica do serviço expõe comportamento sensível ou dados inseguros?
`/usr/local/bin/backup-tool`	Binário privilegiado	Binário setuid detido por root	Qualquer utilizador local o pode invocar com privilégio elevado	Qual é o comportamento real do binário sob semântica setuid?
`/opt/maint/rotate.sh`	Execução agendada	Root, via trabalho cron de 10 em 10 minutos	Escrita livre; qualquer utilizador local o pode modificar	Que ação privilegiada executa o script e como poderia ser abusada?
`/etc/report/report.conf`	Ficheiro sensível em repouso	Ficheiro detido por root	Legível por todos os utilizadores da máquina	São credenciais falsas ou reais, e quanto ganharia um ponto de apoio local?

1.2 Preparação do Lab 1 (P1, P2)

Lab 1, P1. Corri `date; cat /etc/ssc-image-release` antes da sessão. A Figura 1 mostra a captura (imagem, não texto): a data prova que a máquina já estava ligada. É a **SSC lab base image 1.0**, `arch: arm64` — VM **ARM64** em host Apple Silicon via QEMU, cgroup v2 válido.



Figura 1 Lab 1, P1 — `date; cat /etc/ssc-image-release` antes da sessão.

Lab 1, P2. Enumerar a superfície de ataque é listar tudo o que a máquina expõe: o que escuta em rede, o que corre com privilégio, o que corre por calendário e o que está exposto no disco. Faço isto **antes** de explorar, para julgar depois com base no que registei. Estas notas vão diretas para o modelo de ameaças do Lab 2.

2 Lab 2 — Respostas

2.1 Perguntas de preparação (P1, P2)

2.2 P1 — Quais são os quatro elementos de um diagrama de fluxo de dados?

P1. Um DFD tem quatro elementos: **entidades externas** (quem está fora do nosso controlo, ex. cliente na rede, dono), **processos** (código a correr), **armazenamentos** (dados parados) e **fluxos** (setas entre eles). Por cima marco as **fronteiras de confiança**, onde muda o privilégio. É aí que estão as ameaças.

2.3 P2 — O que é que STRIDE dá que uma lista ad-hoc de "problemas" não dá?

P2. STRIDE obriga-me a perguntar as seis categorias (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) a *cada* elemento do DFD. Com uma lista ad-hoc escrevo só o que me lembro e não dá para rever: não sei se "não há ameaça" ou se "não pensei nela". STRIDE não garante modelo completo — ataques por composição escapam na mesma.

3 Parte A — O atacante e os ativos

Primeiro fixo quem ataca e o que interessa proteger. Sem isso o modelo não dá para testar.

3.1 Exercício 1 — Perfil do atacante

1a. O `student` é dono e administrador total da VM. Por isso fica fora do modelo: não vou modelar o dono a atacar a própria máquina. O atacante é um foothold não privilegiado, a conta de serviço **`svc-report`** — a conta por trás do serviço da porta 8081 do Lab 1.

Uso `svc-report` e não "qualquer utilizador": interessa a identidade exata, porque é ela que tem o sudo estreito e que lê `report.conf` através do serviço. "Não ser sudoer geral" e "não ter sudo nenhum" são coisas diferentes, e é isso que meço aqui (ver lista abaixo).

Execução de código local não privilegiada, não dono

Estatuto: geral (fora do escopo `student`; é a posição de partida do modelo).

Não pertence ao grupo `docker`

Estatuto: geral.
Registo: `id svc-report` devolve apenas `gid=988(svc-report)`; `docker:x:1000:student`.

Detém uma concessão sudo passwordless

Estatuto: específico.
Registo: `sudo -l -U svc-report` (Listagem 1).

Lê `report.conf` por herança do serviço

Estatuto: específico.
Registo: `report.service` corre como `svc-report` e lê o ficheiro a cada pedido.

Alcance de rede até a máquina

Estatuto: premissa.
Registo: serviço em `0.0.0.0:8081` medido no Lab 1; alcançabilidade de fora da VM **não provada**.

A conta é de serviço, não de pessoa: `getent passwd svc-report` dá shell `/usr/sbin/nologin` e home `/opt/report`. O foothold é por isso **código a correr como `svc-report`**, não sessão interativa. A última linha abaixo é **premissa**, não medição: no Lab 1 vi `0.0.0.0` mas só testei em `127.0.0.1`, por isso o cliente remoto conta como premissa.

1b. O sudo não dá para adivinhar, tem de se ver na máquina. Corri `sudo -l -U svc-report` e copiei a saída tal como veio:

Listing 1 O que a máquina reporta para `svc-report`: consulta de política, nada é executado.

```
Matching Defaults entries for svc-report on ssc-lab:
    env_reset, mail_badpass, secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin
    :/usr/bin:/sbin:/bin:/snap/bin, use_pty

User svc-report may run the following commands on ssc-lab:
    (ALL) NOPASSWD: /usr/bin/find
```

O ficheiro que produz esta regra é `/etc/sudoers.d/svc-report` (proprietário `root`, modo `0440`), cujo conteúdo inteiro é uma linha:

```
svc-report ALL=(ALL) NOPASSWD: /usr/bin/find
```

Ou seja: **um comando, sem palavra-passe, como qualquer utilizador**, incluindo root. Não é "não ter sudo": é ter uma coisa estreita que alguém achou segura. Registo o alcance como **por medir**.

> Pergunta em aberto, igual à do `backup-tool` no Lab 1. **O que é que este comando dá na prática?** Não assumo que é inofensivo só por ser um comando, e não exploro aqui — isso fica para o Lab 6, Parte D. Até lá fica como folha "adiada" na árvore (Parte D).

1c. **Posição.** Código local como `svc-report` (uid 997), sem grupo `docker`, com `(ALL) NOPASSWD: /usr/bin/find` (alcance por medir, resolve-se na Semana 6), mais alcance de rede — isto último como **premissa**.

Objetivos. (i) **chegar a root**; (ii) **ler o que não deve** — `report.conf` não conta aqui, porque esta conta já o lê legitimamente; contam dados só de root (ex. `/etc/shadow`, que além disso tem `root:*` e por isso só daria divulgação, não root) e ficheiros de outras contas; (iii) **deitar o serviço abaixo**, que sustenta o ativo de disponibilidade e a coluna D do STRIDE.

Motivação. Esta máquina guarda credenciais e dá passagem para o resto: a conta `student` está a um salto (sudo total + grupo `docker`). Parar o serviço de relatórios também é barato para quem já lá está, se quiser perturbar.

3.2 Exercício 2 — Ativos

2a. Ativos por ordem de valor para este atacante (lista abaixo, uma frase cada). A ordem dá a priorização da Parte E: valor do ativo vezes facilidade em lá chegar a partir do foothold.

1. Privilégio de root

Justificação: com root o resto deixa de interessar; é também pré-requisito para ler o que é só de root. O atacante já tem uma via estreita até lá, e fechar esse buraco é o centro do modelo.

2. Integridade do código que root corre sozinho — `/opt/maint/rotate.sh` e, em segundo, `/opt/report/server.py`, que o próprio `svc-report` controla

Justificação: é o caminho mais barato para o n.º 1: 0777, corre como root de 10 em 10 minutos, sem nada por medir.

3. Integridade do sudo de `svc-report` (`/etc/sudoers.d/svc-report`, 0440)

Justificação: é a única capacidade oficial do foothold. Se a regra alargar, o n.º 1 cai sem mais nada.

4. Credenciais em `/etc/report/report.conf`

Justificação: dá leitura direta, mas para *este* atacante vale pouco — ele já lê o ficheiro e as passwords são falsas. Vale como classe de ativo, e vale mais para um utilizador local qualquer, a quem o 0644 entrega tudo.

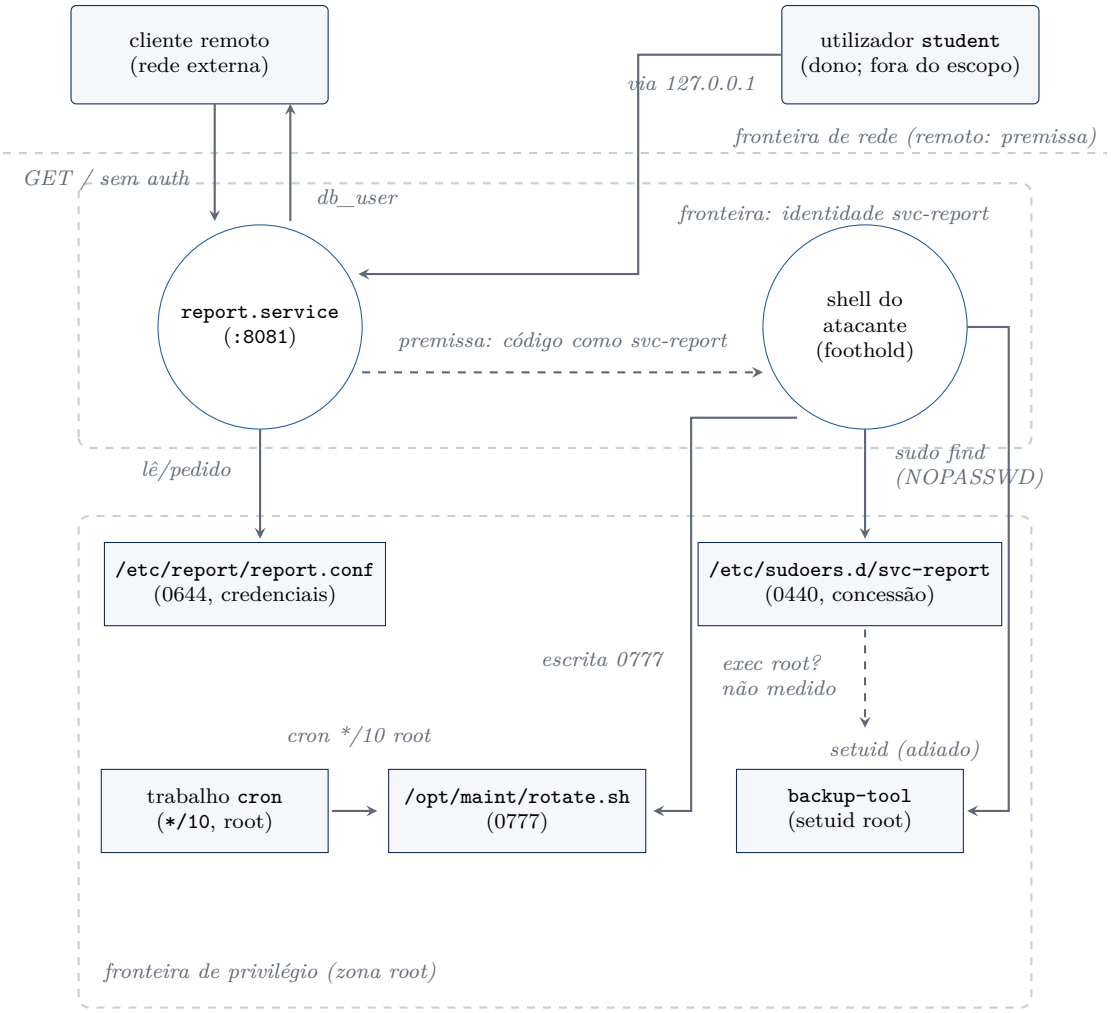
5. Disponibilidade do `report.service` (`:8081`)

Justificação: só conta se o objetivo for perturbar. Não dá privilégio, por isso fica em último.

Notas: as credenciais de `report.conf` são falsas (`db_password=CHANGEME-not-a-real-secret`), mas trato a classe como se fossem reais, que é o que conta numa máquina a sério. Os poderes do `student` não entram como ativo á parte — são o n.º 1 visto do lado do dono, e o dono está fora do modelo.

4 Parte B — Diagrama de fluxo de dados da máquina

O DFD (Exercício 3) passa o inventário do Lab 1 para elementos e fronteiras. Processo e fluxo são coisas separadas: o cron de root é um processo, `cron` → `rotate.sh` é um fluxo. Misturar os dois é o erro que este exercício apanha.



Legenda. Entretangulo = entidade externa; circulo = processo; retângulo com dois traéos = armazenamento de dados; seta contínua = fluxo de dados observado; seta tracejada = fluxo pressuposto ou de alcance não medido; linha tracejada = fronteira de confiança.

Figura 2 DFD da máquina do laboratório com as três fronteiras de confiança (Exercício 3).

3a. A fronteira mais atacada deve ser a **de privilégio, entre o foothold e a zona de root**, não a de rede. A rede só dá o ponto de apoio, que o modelo já assume: sem o segundo salto não vale nada. Os dois ativos de topo (root e o que root corre) estão do outro lado, e há dois cruzamentos abertos: a escrita em `rotate.sh` (nada a trava) e o sudo do `svc-report` (alcance por medir). A fronteira da identidade de serviço também se atravessa, mas com leituras legítimas (`report.conf` é 0644 e o serviço pode lê-lo): está mais aberta do que parece à primeira.

5 Parte C — STRIDE, elemento a elemento

As Tabelas 2 e 3 passam as seis categorias a cada elemento do DFD, incluindo cliente remoto e shell do atacante, para não ficar nada de fora. Em cada célula ou há ameaça concreta ou `n/a` com o porquê. Não preciso de saber explorar — preciso de classificar bem.

Tabela 2 STRIDE aplicado aos elementos do DFD — parte 1 (processos e fluxos).

Elemento	S	T	R	I	D	E
`report.service` (`:8081`) (processo)	Cliente não autentica o servidor (HTTP sem TLS); um processo local pode ocupar a porta e passar-se por ele	O código do serviço (`/opt/report/ser` é escrito pelo próprio uid que o executa: um foothold reescreve o serviço para comportamento futuro	Pedidos não são registados nem atribuíveis	Devolve `db_user` sem autenticação; lê `report.conf` a cada pedido	Processo único; o próprio uid pode matá-lo e derrubar `:8081`	A identidade do serviço deté a concessão sudo: comprometâ-la entrega essa capacidade
Trabalho `cron` como root (processo)	`n/a` — a agenda lê `/etc/cron.d` (root:root), não há identidade a forjar	`n/a` — a agenda em si não é modificável; a adulteração está no fluxo e no armazenamento	`n/a` — execução automática, sem acão de utilizador a negar	`n/a` — a rotina é um placeholder (`:`) e não divulga dados	Um script que não termina bloqueia as execuções seguintes do mesmo trabalho	Corre como root: qualquer código que o cron chegue a correr herda root
`cron` → `rotate.sh` (fluxo)	`n/a` — caminho absoluto fixo, sem identidade em trânsito	Ficheiro 0777: qualquer uid escreve o que root executa (observado no Lab 1)	As escritas não são auditadas: o autor pode negar a alteração	`n/a` — o conteúdo não é segredo	Conteúdo trocado por um script que bloqueia: a manutenção pára	Escrita de código executado por root → escala de privilério (o T habilita o E)
`report.service` → cliente remoto, `db_user` (fluxo)	Sem TLS nem autenticação: cliente e servidor são mutuamente forjáveis	HTTP em texto claro: a resposta é alterável em trânsito	Cliente anónimo e sem logs: o requerente pode negar que pediu os dados	Devolve `db_user=report` sem autenticação (observado no Lab 1)	`n/a` — a exaustão de recursos é do processo, não do fluxo	`n/a` — uma resposta não concede privilégio nesta máquina
`backup-tool` (setuid)	`n/a` — binário não afirma identidade a forjar	`n/a` — root:root 4755, não escrevável por não-dono	`n/a` — não participa em transações	É o `cp` (coreutils 9.4) sob setuid root: pode ler ficheiros só de root por comando do invocante (mecanismo adiado)	`n/a` — invocá-lo não retira disponibilidade	Qualquer utilizador local o invoca com `euid=0`; se aproveitar argumentos do chamador, escala (adiado, Semana 6)

Tabela 3 STRIDE aplicado aos elementos do DFD — parte 2 (armazenamentos, cliente remoto e foothold).

Elemento	S	T	R	I	D	E
`report.conf` (armazenamento)	`n/a` — ficheiro não tem identidade	`n/a` — root:root 0644: escrito só por root	`n/a` — não há transação a negar	0644: credenciais legíveis por qualquer conta local (observado no Lab 1)	`n/a` — armazenamento de leitura, sem estado a esgotar	`n/a` — ler não concede privilégio nesta máquina
`rotate.sh` (armazenamento)	`n/a` — não é um ator	0777: qualquer uid altera o conteúdo (observado)	Sem registo de escrita: o autor nega a alteração	`n/a` — conteúdo placeholder, não é segredo	Substituição por código que bloqueia → manutenção indisponível	É executado por root (cron `*/10`) → escala de privilério
Política sudo: concessão de `svc-report` (armazenamento)	`n/a` — só root escreve; não há política a forjar	`n/a` — root:root 0440	`n/a` — o sudo regista os comandos executados (`use_pty`)	`n/a` — 0440; revelar a concessão ao próprio titular (`sudo -l`) é o intendido	`n/a` — a política não é um serviço a indisponibilizar	`(ALL) NOPASSWD: /usr/bin/find`: comando permitido como root de alcance não medido → possível execução arbitrária (adiado, Semana 6)
Cliente remoto (entidade externa)	Identidade inexistente: qualquer parte se apresenta como cliente; pode ainda passar-se pelo servidor junto de um cliente legítimo	Pedidos e respostas em texto claro, alteráveis em trânsito	Cliente anónimo e sem logs: negação possível	`n/a` — não envia credenciais (não há autenticação), nada seu é divulgado	Pode abrir muitas ligações e esgotar o serviço	`n/a` — não detém privilégio nesta máquina
Shell do atacante / ponto de apoio (processo)	Os processos correm como `svc-report`: indistinguíveis da atividade legítima do serviço	Altera tudo o que `svc-report` escreve: `/opt/report/serv` e o diretório `/opt/report`	As ações ficam atribuídas à conta de serviço ("o serviço fez"), sem separação	Lê tudo o que a identidade lê: `report.conf` (legítimo) e qualquer 0644	Pode matar os processos do serviço (mesmo uid) → derrubar `:8081`	Detém `NOPASSWD` (`/usr/bin/find`) → possível escala a root (alcance não medido)

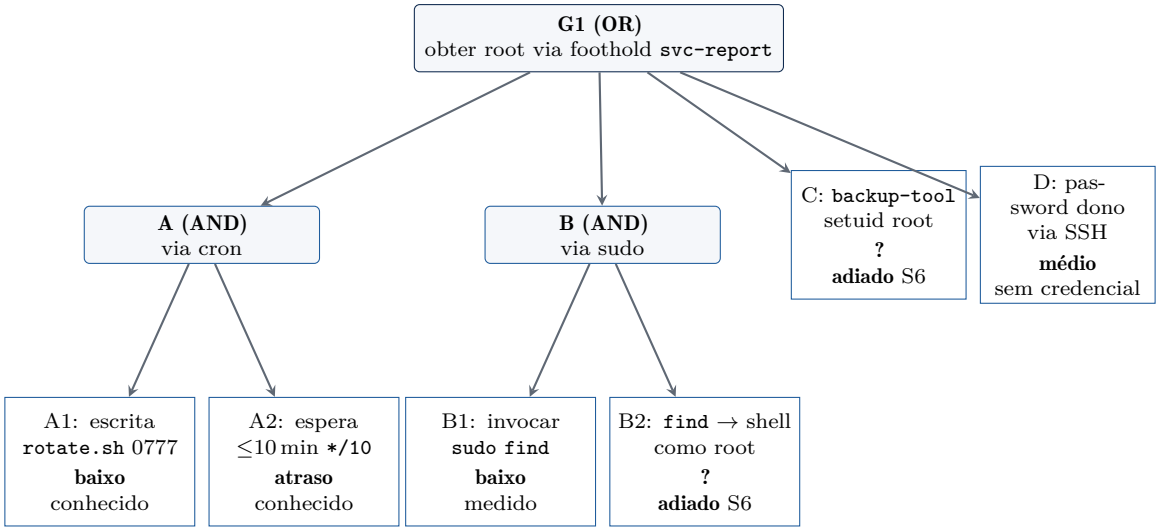
4a. As três piores células:

1. **T** — fluxo `cron` → `rotate.sh` (e o ficheiro): quem não tem privilégio escreve no que root corre de 10 em 10 minutos e fica com código como root. Atraso máximo de dez minutos, dano total.
2. **E** — sudo do `svc-report`: se o `find` der execução arbitrária, é root num comando, sem espera, só com rasto no log do sudo. Mesmo dano, caminho mais curto, sem calendário.
3. **I** — `report.service` → cliente, `db_user`: quem chegar a `:8081` leva material de credencial sem login. Numa máquina a sério era a fuga do segredo que o serviço devia guardar; vi isto no Lab 1.

4b. A que só apanhei por correr a grelha toda foi o **R** no fluxo `cron` → `rotate.sh`: as escritas não têm log, por isso quem escreve pode negar. Por intuição não ia lá: o DFD não diz que não há auditoria, a árvore só quer saber se a escrita funciona, e a lista do Lab 1 só aponta o visível (0777, 0644, setuid). Repúdio só dói depois, quando é preciso provar *quem* fez o quê. A outra que também me passava era o D no `rotate.sh`: trocar por um script que bloqueia pára a manutenção, DoS sem privilégio nenhum.

6 Parte D — Uma árvore de ataque

Objetivo **G1: chegar a root a partir do `svc-report`** (o ponto de partida é sempre o foothold, não o `student`). Nota prévia, confirmada na máquina: root está bloqueado (`/etc/shadow` tem `root:*`, sem hash). Ler `/etc/shadow` é só divulgação, não dá root. Para G1 é preciso *correr código como root* ou *escrever onde root vai agir*.



Leitura dos nós. OR no nó-raiz: basta um dos ramos. **AND** nos ramos A e B: é preciso todos os filhos. Folhas marcadas **adiado** dependem de um mecanismo ainda não ensinado (semântica setuid e o alcance de `find`, Semana 6).

Figura 3 Árvore de ataque para G1, a partir do ponto de apoio `svc-report` (Exercício 5).

5a. O ataque mais barato não é um caminho, é uma **sub-árvore**: em nó AND é preciso todos os filhos, por isso somo custos em vez de escolher o menor. Aqui é o ramo **A**:

1. Escrever o que quero em `/opt/maint/rotate.sh` (0777, qualquer uid escreve; custo baixo);
2. Esperar pelo cron (`*/10`, até dez minutos);
3. O cron corre o script como root → G1 feito.

Os outros não ganham hoje: o B era um passo só, sem espera, *se* B2 se confirmar — mas B2 está em aberto e fica como "adiado", não assumo; o C está todo adiado (setuid, Semana 6); o D precisa de password que não tenho. Resposta: **{A1, A2}**, com a nota de que o B pode vir a ser mais barato.

5b. O ramo A **não depende** de nada adiado: usa só permissões e cron, que já percebo desde o Lab 1. As folhas adiadas só dizem se há algo *ainda mais barato* (sem espera): se B2 ou C se confirmarem, passam para a frente. A incerteza não bloqueia a resposta, só pode melhorá-la.

Se o objetivo fosse G2 (ler `report.conf` com o foothold), a árvore era trivial: o ficheiro é 0644 e o próprio `svc-report` já o lê, por isso qualquer foothold local lê e o custo é zero. Para este atacante G2 não é problema de modo de ficheiro, é de *separação* — era preciso um segredo que só uma identidade mais estreita lesse. O 0644 só é problema para um atacante mais fraco, um utilizador local qualquer. Moral: **com dois labs já dá para ver o essencial** — escrita livre no que root corre, e um sudo por medir. Falta medir duas coisas: o `find` e o `backup-tool` com setuid.

7 Parte E — Priorizar e medir

A Tabela 4 ordena as fraquezas da mais urgente para a menos urgente, com base nas Partes C e D. Cada linha traz descrição, STRIDE, a mudança que resolve e porquê esta ordem: valor do ativo vezes custo para lá chegar.

Tabela 4 Fraquezas da máquina, ordenadas da mais urgente à menos urgente (Exercício 6) — n.º 1 a 3.

#	Fraqueza	STRIDE	Remediação (nomeada)	Justificação do rank
1	`/opt/maint/rotate.sh` 0777, executado por root de 10 em 10 min	T (habilita EoP)	`root:root 0755` (tirar escrita a não-dono)	Caminho mais barato p/ root: 2 passos baratos, nada por medir; alcançável por *qualquer* conta local.
2	`NOPASSWD: /usr/bin/find` p/ `svc-report`	EoP	Remover regra em `/etc/sudoers.d/`	Root num comando, sem espera — mas alcance não medido: gravidade condicional até à Semana 6.
3	`backup-tool` (cp 9.4) setuid root p/ qualquer user	EoP (adiado)	Tirar setuid / remover	Mesma forma "programa permitido" com `euid=0`; semântica S6 pendente: abaixo de 2 demonstradas.

Tabela 5 Fraquezas (cont.) — n.º 4 a 6. Legenda STRIDE: T=Tampering, EoP=Elevation of privilege, I=Information disclosure, S=Spoofing.

#	Fraqueza	STRIDE	Remediação (nomeada)	Justificação do rank
4	`report.conf` 0644 + `db_user` sem auth	I	0600 + não devolver credencial s/ auth	Dano real mas limitado: sementes falsas; o modelado já lê por herança; bandeja p/ user não relacionado.
5	SSH com password auth (`yes`)	S / EoP	`AuthenticationMethods publickey`	Porta de entrada inicial vs dono/adivinha; fora do caminho modelado, mas fecha rota inteira.
6	`/opt/report/server.py` escrito pelo próprio uid	T	`/opt/report` p/ `root:root 0755`	Exige foothold prévio p/ persistir: por isso em último.

6a. O 1 vem antes do 2 por duas razões. Primeiro, **chega mais gente lá**: o 1 dá para qualquer conta local, o `find` só para o `svc-report` — e ambos dão root. Segundo, **o 1 está provado**: 0777 e cron como root, medidos no Lab 1, sem nada por explicar. O 2 depende de B2, que ainda não sei. Fechar o 1 tira hoje um caminho certo para root; fechar o 2 pode ser fechar algo inofensivo, se B2 for negativo. E mesmo que B2 seja positivo, o 2 ainda exige o foothold `svc-report`, o 1 não exige nada.

6b. Métrica: **número de ficheiros com escrita livre que root corre**. Conta-se com um comando e apanha o problema do Lab 1 (quem escreve *vs* quem corre). Hoje vale **1**: só o `/opt/maint/rotate.sh` sai do `find /opt /etc /usr/local /var/spool -xdev -perm -0002 -type f`. Com o fix do item 1 passa a **0**.

Contra-exemplo para mostrar que a métrica interessa: "serviços de rede como root" vale hoje **1** (o `sshd` na 22; o `report.service` é `svc-report`) e **continuava 1** depois do fix — ou seja, não media o efeito da prioridade 1. Métrica só presta se mede o que priorizei.

6c. A política por trás do fix de topo, no formato da aula:

```
Nenhum utilizador não privilegiado pode modificar nada o que o root execute.
```

Dois mecanismos dão isto: (M1) **permissão** — `rotate.sh` para `root:root 0755`; e (M2) **apagar** — tirar o cron e o script, não corre mais nada por ali.

Diferença: M1 mantém a manutenção a correr, mas fico a confiar que ninguém volta a partir o modo; M2 acaba com essa confiança, mas perco a função. A política vale mais que cada fix:

proíbe o próximo caso igual — amanhã um script novo no cron — enquanto M1 só trata deste ficheiro e M2 só trata deste programa.